Aluno: Urbano Coutinho Neto
Ra: 324133695
Github Com os codigos: https://github.com/URBANO2O/CentralInteligenteChamados.git

ATIVIDADE PRÁTICA DE TABELAS HASH E HEAPS EM JAVA

ETAPA 1 - COMPREENDER O PROBLEMA

1. Como localizar um chamado sem percorrer todos os registros?

Resposta: Utilizando uma tabela hash, que permite localizar um chamado pelo seu identificador de forma rápida.

2. Como selecionar o chamado mais urgente?

Resposta: Utilizando um heap máximo, que mantém o chamado de maior prioridade no topo.

3. Como desempatar prioridades iguais?

Resposta: Quando dois chamados possuem a mesma prioridade, o chamado que chegou primeiro deve ser atendido primeiro.

4. Por que uma ArrayList isolada não é a melhor escolha?

Resposta: Porque a busca por identificador pode exigir percorrer todos os elementos, e encontrar o chamado mais urgente também pode exigir verificar a lista inteira.

ETAPA 5 - EXPLICAR AS IMPLEMENTAÇÕES

Dupla A - Tabela Hash

O índice é calculado usando o identificador do chamado e o tamanho da tabela. As colisões acontecem quando identificadores diferentes ficam na mesma posição. Utilizamos listas encadeadas para guardar todos os chamados sem perder informações. Os métodos permitem cadastrar, buscar e remover chamados.

Dupla B - Heap Máximo

A subida acontece quando um chamado de maior prioridade precisa subir no heap. A descida acontece após uma remoção para reorganizar os elementos. Quando dois chamados possuem a mesma prioridade, o que chegou primeiro fica na frente.

ETAPA 8 - REGISTRO DOS TESTES

Caso 1 - Cadastro

Entrada: Cadastro de seis chamados.

Resultado obtido: Todos os chamados foram cadastrados corretamente.

Caso 2 - Colisão

Entrada: Cadastro dos identificadores 10, 17 e 24.

Resultado obtido: Os três chamados foram mantidos e podem ser consultados.

Caso 3 - Busca

Entrada: Pesquisa pelo identificador 24.

Resultado obtido: O chamado correto foi encontrado.

Caso 4 - Busca inválida

Entrada: Pesquisa pelo identificador 999.

Resultado obtido: O sistema informou que o chamado não foi encontrado.

Caso 5 - Prioridade e empate

Entrada: Atendimento de chamados com prioridade igual a 5.

Resultado obtido: O chamado que chegou primeiro foi atendido antes.

Caso 6 - Fila vazia

Entrada: Tentativa de atender quando não existem chamados.

Resultado obtido: O sistema informou que não existem chamados para atender.

Observação: Os resultados devem ser conferidos na execução do programa antes de marcar os testes como aprovados.

ETAPA 9 - REVISÃO POR PARES

Ponto positivo:

O sistema organiza os chamados por identificador e prioridade, permitindo buscas rápidas e atendimentos na ordem correta.

Sugestão de melhoria:

Criar um menu para facilitar o cadastro, a pesquisa e o atendimento dos chamados.

ETAPA 10 - QUESTÃO FINAL

Por que a solução utiliza uma tabela hash e um heap em vez de apenas uma ArrayList?

Resposta: Porque a tabela hash permite encontrar chamados pelo identificador rapidamente, enquanto o heap máximo organiza os chamados por prioridade e permite atender primeiro o mais urgente.

ETAPA 13 - EXIT TICKET INDIVIDUAL

1. Por que o sistema utilizou duas estruturas?

Resposta: Porque a tabela hash é utilizada para buscar chamados pelo identificador, enquanto o heap máximo organiza os chamados por prioridade.

2. Qual teste revelou um erro ou comportamento importante?

Resposta: O teste de empate mostrou que, quando dois chamados possuem a mesma prioridade, o mais antigo deve ser atendido primeiro.

3. Qual seria o impacto de muitas colisões?

Resposta: As buscas ficariam mais lentas, porque seria necessário verificar mais chamados armazenados na mesma posição da tabela.

4. Que melhoria você implementaria?

Resposta: Implementaria um menu interativo para permitir cadastrar, pesquisar, consultar e atender chamados de maneira mais fácil.